Develop a Library Management System using Hibernate ORM, JPA Annotations, Entity Mapping, SessionFactory, and Hibernate Query Language (HQL).

Hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "https://hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>

        <property name="hibernate.connection.driver_class">
            com.mysql.cj.jdbc.Driver
        </property>

        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/students
        </property>

        <property name="hibernate.connection.username">
            root
        </property>

        <property name="hibernate.connection.password">
            root
        </property>

        <property name="hibernate.hbm2ddl.auto">
```

```
        update
    </property>

    <property name="hibernate.show_sql">
        true
    </property>

    <mapping class="com.og.demoproject3.Book"/>

  </session-factory>
</hibernate-configuration>
```

Book.java

```
package com.og.demoproject3;

import jakarta.persistence.*;

@Entity
public class Book {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;

  private String title;
  private String author;

  public Book() {}

  public Book(String title, String author) {
```

```
        this.title = title;
        this.author = author;
    }
    @Override
    public String toString() {
        return "ID: " + id
                + " | Title: " + title
                + " | Author: " + author;
    }
}
```

DemoProject3.java

```
package com.og.demoproject3;
import org.hibernate.*;
import org.hibernate.cfg.Configuration;
import java.util.Scanner;
import java.util.List;

public class Demoproject3 {

    public static void main(String[] args) {

        SessionFactory factory = new Configuration()
                .configure()
                .addAnnotatedClass(Book.class)
                .buildSessionFactory();

        Scanner sc = new Scanner(System.in);

        try {
```

```
while (true) {
    System.out.println("\n1. Add Book");
    System.out.println("2. View Books");
    System.out.println("3. Search Book");
    System.out.println("4. Exit");
    System.out.print("Enter choice: ");

    int ch = sc.nextInt();
    sc.nextLine();

    if (ch == 4) break;

    try (Session session = factory.openSession()) {

        if (ch == 1) {
            System.out.print("Title: ");
            String title = sc.nextLine();

            System.out.print("Author: ");
            String author = sc.nextLine();

            Transaction tx = session.beginTransaction();
            session.persist(new Book(title, author));
            tx.commit();

            System.out.println("Book added!");

        } else if (ch == 2) {
            List<Book> books = session
```

```
                    .createQuery("FROM Book", Book.class)
                    .list();

                books.forEach(System.out::println);

            } else {
                System.out.println("Invalid choice!");
            }
        }
    }
    } finally {
        sc.close();
        factory.close();
    }
  }
}
```